

Article

Temporally-Aware Deep Reinforcement Learning for Dynamic Obstacle Avoidance in UAVs

Chang Liu and Shan Wang *

School of Artificial Intelligence, China University of Mining and Technology-Beijing, Beijing 100083, China; liuchang@student.cumtb.edu.cn

* Correspondence: wangshan@cumtb.edu.cn

Highlights

What are the main findings?

- A compact two-frame multi-layer light detection and ranging (LiDAR) representation and a convolutional neural network–long short-term memory (CNN-LSTM) recurrent proximal policy optimization (Recurrent PPO) policy are developed to extract local geometric structures and short-term dynamic cues for unmanned aerial vehicle (UAV) obstacle avoidance.
- A velocity-projection action shield is introduced to correct high-risk policy outputs during training and execution, and simulation results show an average success rate of 91.88%, with an average online computation time of 0.78 ms across four test configurations.

What are the implications of the main findings?

- Lightweight temporal LiDAR observations can improve local navigation and short-term collision awareness without explicit obstacle tracking or trajectory prediction.
- The proposed framework provides a computationally efficient obstacle-avoidance strategy that reduces collision risk for small UAVs in mixed static–dynamic environments.

Abstract

Autonomous obstacle avoidance for UAVs in dynamic obstacle-dominated complex environments must address time-varying local collision risks from multiple directions under the constraints imposed by local sensing, environmental uncertainty, execution safety, and limited onboard computation. Planning-based methods often require frequent replanning or explicit obstacle prediction, whereas conventional reinforcement learning policies may produce myopic decisions under partial observability. To address these limitations, this study proposes a dynamic obstacle-avoidance framework that combines a temporal LiDAR representation with safety-aware action correction in recurrent reinforcement learning. Multi-layer LiDAR observations are constructed using sector-wise minimum pooling. Adjacent two-frame stacking and a CNN-LSTM architecture are then used to extract local geometric structures and short-term dynamic cues, and a velocity-control policy is optimized using Recurrent PPO. In addition, a smooth velocity-projection safety shield is introduced to modify policy outputs and reduce collision risk during both training and policy execution. Experiments conducted in mixed static–dynamic obstacle scenarios based on Gym-PyBullet-Drones show that the proposed method achieves an average success rate of 91.9% across four test configurations, with an average online computation time of 0.78 ms. Ablation studies further support the contributions of two-frame observations, LSTM-based temporal modeling, and the safety shield.



Academic Editors: Salvatore Rosario Bassolillo, Vito Antonio Nardi and Giuseppe Tricomi

Received: 28 May 2026

Revised: 28 June 2026

Accepted: 30 June 2026

Published: 3 July 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and

conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

Keywords: unmanned aerial vehicles; dynamic obstacle avoidance; temporal LiDAR representation; deep reinforcement learning; safety shield

1. Introduction

Unmanned aerial vehicles (UAVs), particularly multirotor platforms, have been widely used in civilian applications, such as forest-fire monitoring, warehouse inspection, and express delivery, owing to their high maneuverability and cost-effectiveness [1]. In practical applications, UAVs are often required to navigate autonomously and avoid obstacles in unknown or partially known environments that may contain both static and moving objects. In this study, complex dynamic environments refer to local mixed static–dynamic obstacle-avoidance scenarios in which nearby moving obstacles continuously change the local collision-risk distribution perceived by the UAV. Compared with static-obstacle environments, these scenarios are more challenging because the UAV must perceive local obstacle geometry, capture short-term collision cues, and generate safe control commands in real time under limited onboard computational resources and control-cycle constraints. Consequently, dynamic obstacle perception, real-time planning, execution safety, and computational efficiency remain central challenges for autonomous UAV navigation in such environments [1–5].

Existing UAV obstacle-avoidance methods can be broadly divided into two categories: planning-based and optimization-based approaches, and learning-based approaches. Planning-based and optimization-based methods offer good interpretability and have been widely used in engineering systems [4–9]. However, when applied to dense or rapidly changing dynamic scenes, these methods often require frequent replanning, explicit obstacle-state estimation, or multi-stage perception–prediction–planning pipelines, which increases system complexity and online computational cost. Learning-based methods can generate control commands from local sensor observations with relatively low inference latency [10–17]. Nevertheless, feedforward policies are often limited in their ability to encode obstacle-motion trends under partial observability, which may lead to myopic decisions and increased collision risk [18–20]. Therefore, achieving real-time UAV dynamic obstacle avoidance with lightweight local observations and reduced collision risk remains a challenging problem.

To address the above challenge, this study develops a temporally-aware deep reinforcement learning framework for UAV dynamic obstacle avoidance by integrating a two-frame multi-layer light detection and ranging (LiDAR) representation, convolutional neural network–long short-term memory (CNN-LSTM) feature extraction, recurrent proximal policy optimization (Recurrent PPO), and velocity-projection action correction. Compact and conservative multi-layer LiDAR range observations are constructed through sector-wise minimum pooling. Two-frame stacking and LSTM hidden states are then combined to capture short-term dynamic variations from consecutive local observations. In this study, temporal awareness refers to the use of short-term temporal information from consecutive LiDAR observations and recurrent hidden states, rather than relying only on a single instantaneous range observation. Specifically, the two-frame LiDAR stack provides explicit short-term range changes, while the LSTM module aggregates recent observation history to support local decision-making under partial observability. At the control level, Recurrent PPO is used to learn a velocity-control policy, while a velocity-projection safety shield is applied during both training and execution to correct high-risk policy outputs, reduce collision risk, and encourage lower-risk avoidance behaviors. The aim of this study is to develop and evaluate the proposed framework for UAV dynamic obstacle avoidance

under controlled simulation conditions. Training and testing are conducted in simulation, and real-world sensor noise and hardware-related uncertainties are not explicitly modeled.

The main contributions of this study are summarized as follows:

- A lightweight temporal LiDAR representation is designed for UAV dynamic obstacle avoidance. By combining sector-wise minimum pooling, two-frame stacking, and low-dimensional navigation features, the proposed representation preserves local obstacle geometry and short-term dynamic cues while maintaining a compact input dimension.
- A CNN-LSTM-based recurrent policy is trained using Recurrent PPO for UAV velocity control in partially observable dynamic environments. The two-frame observation provides explicit short-term variation information, and the LSTM hidden state further encodes historical observation dependencies, thereby alleviating the myopic behavior of single-frame feedforward policies.
- A velocity-projection action shield is incorporated into both training and execution. The shield smoothly corrects high-risk actions generated by the policy and introduces the safety-shield correction reward into the reward function, which helps reduce collision risk in the tested simulation scenarios and encourages the policy to generate fewer high-risk action outputs.

2. Related Work

2.1. Conventional Obstacle-Avoidance Methods

Conventional UAV obstacle-avoidance methods mainly include graph-search or sampling-based path planning, local reactive avoidance, and optimization-based trajectory planning. Graph-search and sampling-based planners, such as A* [21] and a rapidly exploring random tree (RRT) [22], can generate feasible paths when environmental maps are available, but their direct responsiveness to unknown dynamic obstacles is limited. Artificial potential field (APF) [23] provides a simple reactive mechanism for local obstacle avoidance with low computational cost, but it is prone to local minima in complex environments. Velocity obstacle (VO) [24] characterizes dynamic collision risk in relative-velocity space, but it usually requires explicit estimation of obstacle states, including position and velocity.

In recent years, optimization-based trajectory planning has been widely used for quadrotor obstacle avoidance. For example, EGO-Planner [6] reduces the computational burden of local trajectory optimization by avoiding full Euclidean signed distance field (ESDF) construction, thereby improving real-time planning performance and engineering applicability. FASTER [9] maintains flight safety by retaining a safe backup trajectory in known free space. For dynamic obstacles, ViGO [7] tracks and geometrically represents dynamic objects in a voxel map using an onboard depth camera, and combines this representation with gradient-based B-spline trajectory optimization to evaluate dynamic collision risk through a receding-horizon distance field. FAPP [8] integrates point-cloud segmentation, dynamic object association, adaptive motion estimation, trajectory optimization, and replanning to handle cluttered dynamic environments. Intent-prediction-driven model predictive control (MPC) methods [4] further model the motion intent of dynamic obstacles probabilistically, generate trajectory predictions, and incorporate intent probabilities into MPC trajectory scoring to improve safety in human–robot mixed scenarios.

Despite these advances, planning-based methods usually require several coupled modules. In environments with multiple moving obstacles, perception errors, prediction uncertainty, and accumulated inter-module errors may degrade avoidance performance. Moreover, for small UAVs with limited onboard computation, producing real-time avoidance decisions without explicitly detecting, tracking, and predicting each dynamic obstacle remains an important and practically relevant problem.

2.2. Learning-Based Obstacle-Avoidance Methods

Learning-based methods employ neural networks to map sensor observations to control commands and usually incur low online inference cost during deployment [13]. The success of deep Q-network (DQN) [16] demonstrated the ability of deep reinforcement learning to extract decision-relevant features from high-dimensional inputs, which promoted subsequent applications in navigation, obstacle avoidance, and target tracking [10–15,17,25,26].

Compared with visual sensors, LiDAR provides direct range measurements and is less sensitive to illumination changes, making it particularly practical for obstacle avoidance [1]. However, raw point clouds are high-dimensional, variable in size, and structurally irregular; using them directly for policy learning increases the complexity of the policy network and the training process. Consequently, LiDAR-based learning studies often transform point clouds into task-oriented representations. Some methods compress raw point clouds into low-dimensional representations suitable for reinforcement learning while preserving key geometric information, such as small-obstacle cues [25,27]. Others project and compress point clouds into two-dimensional range maps to facilitate feature extraction by deep neural networks [11,26,27]. These studies indicate that lightweight and informative LiDAR representations are important for learning-based obstacle avoidance on UAV platforms with limited computation and sensing resources.

2.3. Temporal Modeling and Action Filtering in Reinforcement Learning

Dynamic obstacle avoidance is inherently partially observable: a single observation captures only the current local range distribution and provides limited information about obstacle-motion trends and short-term collision risk. Recurrent neural networks have been shown to mitigate partial observability in deep reinforcement learning [19]. In applied obstacle-avoidance studies, temporal-memory mechanisms have been used to exploit historical observations and improve adaptability to changing environments in obstacle-avoidance tasks involving unmanned surface vehicles and UAVs [28,29]. Memory-based deep reinforcement learning has also been applied to monocular-image-based navigation, where a deep recurrent Q-network with temporal attention is used to process image-derived depth observations for UAV obstacle avoidance [20]. For LiDAR-based mobile-robot navigation, spatiotemporal attention has been combined with a 2D LiDAR descriptor to highlight dynamic obstacles without explicit object tracking [18]. These studies suggest that temporal processing can improve collision-avoidance decision-making based on sequential sensor observations.

In addition, reinforcement learning policies are usually represented by black-box neural networks, which makes execution safety difficult to guarantee. To improve safety, previous studies have introduced action-filtering or safety-layer mechanisms at the policy output to correct high-risk actions [10,30].

Motivated by the above observations, this study addresses dynamic obstacle avoidance for small quadrotors using a two-frame sector-pooled LiDAR representation, CNN-LSTM temporal feature extraction, and a safety shield that smoothly corrects high-risk velocity commands. Unlike modular approaches that require explicit perception, tracking, and prediction of individual dynamic obstacles [3,4], the proposed method does not explicitly model obstacle trajectories. Instead, it learns a policy mapping from compact local LiDAR observations and self-state information to velocity commands, with a lightweight safety shield applied as post-policy correction.

3. Materials and Methods

This study adopts a learning-based local obstacle-avoidance methodology, in which the UAV generates velocity commands online from compact onboard LiDAR observations.

The proposed framework uses established components, including the CNN module for spatial feature extraction, LSTM for temporal feature aggregation, and PPO for policy learning. For local dynamic obstacle avoidance, these components are configured and integrated within a task-specific framework that combines a two-frame multi-layer LiDAR range representation, a CNN-LSTM recurrent policy trained using Recurrent PPO, and a velocity-projection safety shield. The two-frame representation provides compact nearest-obstacle information and short-term range variation cues, while the recurrent policy supports local decision-making under partial observability. The velocity-projection safety shield corrects high-risk velocity commands during both training and execution and incorporates the correction magnitude into the reward function. Compared with single-frame feedforward reinforcement learning (RL) policies, this combination introduces short-term temporal perception and recurrent memory into the local avoidance policy. Compared with purely learned action outputs, the safety shield provides an additional action-level correction mechanism for potentially risky velocity commands. Through this design, the framework is intended to improve local dynamic obstacle-avoidance reliability in the tested simulation scenarios without requiring explicit obstacle tracking, trajectory prediction, or global replanning during online execution.

3.1. Problem Formulation

This study considers the dynamic obstacle-avoidance task of a single quadrotor UAV in a bounded workspace. The UAV is required to navigate from a start position to a goal position while avoiding static obstacles, dynamic obstacles, and boundary fences. The UAV primarily performs horizontal motion, while limited altitude variation is allowed to maintain flight within the prescribed altitude range.

Since the policy cannot access the full states of all obstacles, the task is modeled as a partially observable Markov decision process (POMDP). The policy makes decisions based on local observations, including onboard LiDAR measurements, UAV velocity, the relative goal position, the previous action, and an auxiliary navigation feature vector. The observation at time step t is defined as follows, where t denotes one high-level policy decision step. The PyBullet simulation and the low-level proportional-integral-derivative (PID)-based flight controller operate at 240 Hz, while each high-level policy action is applied for eight simulation steps. Therefore, consecutive policy decision steps are separated by $8/240 = 1/30$ s, corresponding to a policy frequency of 30 Hz:

$$o_t = [L_t, L_{t-1}, \mathbf{p}_{g,t}^{\text{rel}}, \mathbf{v}_t, \mathbf{a}_{t-1}, \mathbf{m}_t], \quad (1)$$

where o_t denotes the observation vector, L_t and L_{t-1} denote the current and previous LiDAR range observations, respectively, $\mathbf{p}_{g,t}^{\text{rel}}$ denotes the goal-relative position, $\mathbf{v}_t$ denotes the UAV velocity, $\mathbf{a}_{t-1}$ denotes the previous action, and $\mathbf{m}_t$ denotes an auxiliary navigation feature vector derived from the current LiDAR observation and the relative goal direction. The detailed construction of $\mathbf{m}_t$ is described in Section 3.3. The goal-relative position is defined as

$$\mathbf{p}_{g,t}^{\text{rel}} = \mathbf{p}_g - \mathbf{p}_t, \quad (2)$$

where $\mathbf{p}_g$ denotes the goal position, and $\mathbf{p}_t$ denotes the UAV position at time step t .

The local observation does not include absolute altitude or the attitude quaternion as separate entries. Absolute altitude is not explicitly provided because the task is conducted within a narrow constrained altitude range, while the vertical information required for navigation is retained through the relative goal altitude in $\mathbf{p}_{g,t}^{\text{rel}}$, the vertical velocity component in $\mathbf{v}_t$, and the multi-layer LiDAR observations. The policy outputs a normalized continuous action, which is converted into a velocity command containing horizontal

motion components and a vertical altitude-regulation component before being tracked by the low-level PID-based flight controller. The attitude quaternion is therefore not included in the high-level local observation, because attitude stabilization and command tracking are handled by the low-level controller using the required vehicle-state information, including orientation. An episode ends when the UAV reaches the goal, collides with an obstacle or a boundary fence, or exceeds the maximum number of allowed steps, which is set to 1000. The safety-shield mechanism and reward design are detailed in Sections 3.6 and 3.7, respectively.

3.2. LiDAR State Representation

Before being fed into the policy network, the raw LiDAR range measurements are compressed into a compact multi-layer range representation through sector-wise minimum pooling. Here, sector-wise minimum pooling refers to dividing each LiDAR pitch layer into fixed horizontal sectors and retaining the minimum range value within each sector. This operation preserves the nearest-obstacle information in a compact and conservative form. The onboard LiDAR performs horizontal 360° scanning at five pitch layers, with 216 raw rays uniformly emitted in each layer. To reduce the input dimension, each pitch layer is uniformly divided into 72 angular sectors, and each sector contains three adjacent rays. The minimum range measurement within each sector is selected as the sector-level range value. After this operation is independently applied to all pitch layers, each LiDAR frame contains 360 range values. With two-frame stacking, the resulting LiDAR tensor has a shape of $2 \times 5 \times 72$, corresponding to 720 range values.

Sector-wise minimum pooling compresses the LiDAR observation while preserving the nearest-obstacle information in each sector. Unlike average pooling, which may weaken the response of small or nearby obstacles, minimum pooling retains the nearest-obstacle range in each sector, providing a more conservative representation for collision avoidance. The use of a compact LiDAR representation is motivated by prior LiDAR-based learning studies, in which sensor inputs are compressed to reduce reinforcement learning complexity while preserving geometric information relevant to navigation and collision avoidance [11,25,26].

Figure 1 visualizes the resulting two-frame LiDAR range representation.

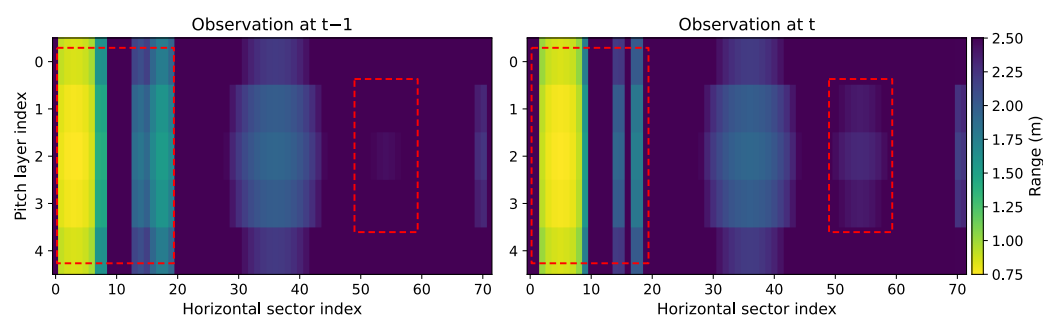


Figure 1. Two-frame (5×72) LiDAR range observation heatmap. Each frame contains five pitch layers and 72 horizontal sectors, and brighter colors indicate shorter obstacle ranges. The red dashed boxes highlight representative sectors with short-range obstacles and local temporal range variations between the previous frame and the current frame. By stacking the previous and current LiDAR observations, local range-distribution changes between adjacent time steps are represented explicitly, providing short-term temporal cues for dynamic obstacle avoidance.

3.3. Overall Framework

Figure 2 illustrates the overall framework of the proposed method.

The two-frame LiDAR range representation described in Section 3.2 is first processed by a two-dimensional CNN [31]. In this framework, the CNN is used to extract local spatial and geometric features from the $2 \times 5 \times 72$ LiDAR range tensor before temporal

aggregation by the LSTM. Since the horizontal LiDAR dimension is periodic, whereas the pitch dimension is not, circular padding is applied along the horizontal direction and zero padding is applied along the pitch direction during convolution. The CNN encoder consists of two convolutional layers with 32 and 64 channels, respectively. The kernel sizes of the two convolutional layers are 3×5 and 3×3 , respectively, and both layers use a stride of (1, 2).

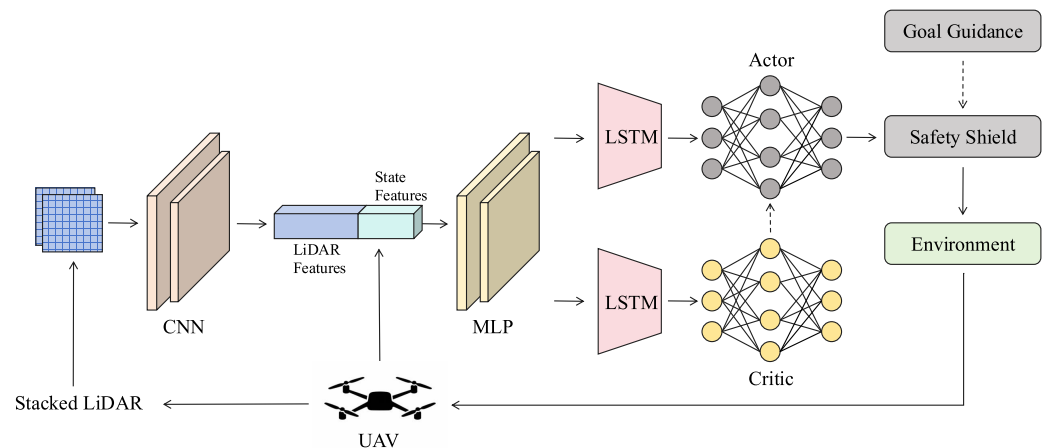


Figure 2. Overall framework of the proposed method. The stacked two-frame LiDAR observations are processed by a CNN to extract local range features, which are combined with UAV state and navigation features and encoded by a MLP. The fused representation is then fed into the LSTM-based recurrent module for temporal feature aggregation. Based on the recurrent features, the Actor outputs a continuous velocity-control action, while the Critic estimates the state value for Recurrent PPO training. The goal-guidance term provides auxiliary task-direction information during early-stage training, and the safety shield corrects potentially risky actions before execution in the environment.

The extracted spatial features are then flattened and concatenated with the relative goal position, UAV velocity, previous action, and an auxiliary navigation feature vector. The auxiliary navigation features are derived from the current LiDAR observation and the relative goal direction, including goal-direction clearance, left/right clearance, vertical-channel clearance, the goal-direction blockage flag, and the goal-pitch-angle error. These features provide supplementary information about local traversability without relying on explicit obstacle detection, tracking, or global map information.

The fused feature vector is mapped by a two-layer multi-layer perceptron (MLP) and then passed to the LSTM [32]. The LSTM hidden state encodes historical observation information and captures short-term temporal variations in consecutive LiDAR observations. Based on the temporal features, the Actor branch outputs a normalized continuous action containing horizontal motion components and an altitude-regulation component, while the Critic branch estimates a scalar state value for Recurrent PPO updates. In the implemented network, the post-MLP feature dimension, the LSTM hidden dimension, and the hidden-layer dimensions of both the Actor and Critic heads are set to 256.

Based on the implemented CNN-LSTM Actor-Critic architecture, the complete Actor-Critic network contains 2,749,223 trainable parameters, corresponding to approximately 10.49 MiB of parameter storage in a 32-bit floating-point format. During online execution, only the Actor path is required for action generation, which contains 2,156,838 parameters and requires approximately 8.23 MiB of parameter storage in a 32-bit floating-point format. The Actor LSTM hidden and cell states require approximately 2 KiB of additional memory. A deterministic Actor forward pass requires approximately 4.0 million multiply-accumulate operations, equivalent to approximately 7.9 million floating-point operations when one multiplication and one addition are counted separately.

Before being executed, the action generated by the Actor is further processed to improve exploration efficiency during early training and reduce near-obstacle collision risk during both training and execution. A goal-guidance term is used only during early training to provide auxiliary velocity guidance toward the goal. The safety shield then corrects potentially risky velocity components according to local LiDAR ranges. The corrected velocity command is finally sent to the low-level flight controller for execution.

3.4. Recurrent PPO

PPO [33] constrains the policy update magnitude by optimizing a clipped surrogate objective. In this study, Recurrent PPO is adopted to train the Actor and Critic networks. Owing to the LSTM module, the policy depends not only on the current observation o_t but also on the recurrent hidden state h_{t-1} , which encodes historical observation information. The clipped surrogate objective for policy optimization is written as

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t [\min(\rho_t(\theta)\hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \varepsilon, 1 + \varepsilon)\hat{A}_t)], \quad (3)$$

where $L^{\text{CLIP}}(\theta)$ denotes the clipped surrogate objective, θ denotes the current policy parameters, $\mathbb{E}_t$ denotes the empirical expectation over time steps, $\rho_t(\theta)$ denotes the probability ratio, $\hat{A}_t$ denotes the generalized advantage estimate, ε denotes the PPO clipping coefficient, and $\text{clip}(\cdot)$ denotes the clipping operator. The probability ratio is defined as

$$\rho_t(\theta) = \frac{\pi_{\theta}(\mathbf{a}_t \mid o_t, h_{t-1})}{\pi_{\theta_{\text{old}}}(\mathbf{a}_t \mid o_t, h_{t-1})}, \quad (4)$$

where π_{θ} denotes the current policy, $\pi_{\theta_{\text{old}}}$ denotes the old policy before the current PPO update, θ_{old} denotes the corresponding old policy parameters, $\mathbf{a}_t$ denotes the action at time step t , o_t denotes the observation at time step t , and h_{t-1} denotes the recurrent hidden state from the previous time step. The Critic network is trained using the value-function loss, and an entropy regularization term is included to encourage exploration. The overall optimization objective follows the standard PPO formulation, combining the clipped policy objective, value-function loss, and entropy bonus.

3.5. Curriculum Learning and Goal-Guided Action Fusion

In the early stage of reinforcement learning, purely random exploration often leads to inefficient behaviors, such as detours, oscillations, and frequent collisions. Previous studies have shown that gradually increasing task difficulty through curriculum learning can improve learning efficiency when complex tasks are learned from scratch [34]. To improve training stability and sample efficiency, this study adopts a curriculum learning strategy together with an early-stage goal-guidance term. Here, curriculum learning refers to a training strategy in which the task difficulty is progressively increased, allowing the policy to learn simpler navigation behaviors before being exposed to more difficult mixed static-dynamic obstacle scenarios. The curriculum gradually increases the task difficulty during training by exposing the policy from simpler obstacle configurations to more complex static-dynamic mixed environments. In this way, the policy first learns basic goal-directed flight and obstacle-avoidance behaviors and is then trained under more challenging dynamic conditions. Meanwhile, the goal-guidance term provides auxiliary velocity guidance toward the goal during early training to reduce inefficient exploration when the policy is still insufficiently trained. The influence of this guidance term is gradually weakened during training and is not used during testing.

Let the normalized policy action after action-space clipping be $\mathbf{a}_t = [a_{t,x}, a_{t,y}, a_{t,z}]^T$, with $\mathbf{a}_t \in [-1, 1]^3$. In the implementation, the received action is first clipped to this range

and then converted into a velocity command by applying the maximum command velocity and the vertical action-scaling coefficient. The maximum command velocity is $v_{\max}$, and the vertical action-scaling coefficient is s_z . The policy velocity before speed normalization is defined as

$$\tilde{\mathbf{v}}_{\pi,t} = v_{\max} [a_{t,x}, a_{t,y}, s_z a_{t,z}]^T. \quad (5)$$

To ensure that the command velocity does not exceed the maximum speed, the policy velocity is normalized as

$$\mathbf{v}_{\pi,t} = \begin{cases} \tilde{\mathbf{v}}_{\pi,t}, & \|\tilde{\mathbf{v}}_{\pi,t}\|_2 \leq v_{\max}, \\ v_{\max} \frac{\tilde{\mathbf{v}}_{\pi,t}}{\|\tilde{\mathbf{v}}_{\pi,t}\|_2}, & \|\tilde{\mathbf{v}}_{\pi,t}\|_2 > v_{\max}. \end{cases} \quad (6)$$

The normalized goal-direction vector is computed as

$$\mathbf{e}_{g,t} = \frac{\mathbf{p}_{g,t}^{\text{rel}}}{\|\mathbf{p}_{g,t}^{\text{rel}}\|_2 + \varepsilon_g}, \quad (7)$$

where ε_g is a small positive constant used to avoid division by zero. The goal-guided velocity is then defined as

$$\mathbf{v}_{g,t} = k_g v_{\max} \mathbf{e}_{g,t}, \quad (8)$$

where $k_g = 0.45$ is the guidance gain. The candidate velocity after goal-guided action fusion is given by

$$\mathbf{v}_t^0 = \beta \mathbf{v}_{\pi,t} + (1 - \beta) \mathbf{v}_{g,t}, \quad (9)$$

where $\beta \in [0, 1]$ is the policy-weight coefficient. During training, β is gradually increased so that the influence of the goal-guidance term decreases over time. When $\beta = 1$, the candidate velocity is fully determined by the learned policy, and the goal-guidance term is disabled during testing. The action-fusion parameters, including the goal-guidance gain k_g , the vertical action-scaling coefficient s_z , and the policy-weight schedule β , were set through preliminary simulation-based tuning to stabilize early exploration and maintain feasible velocity commands. The resulting candidate velocity $\mathbf{v}_t^0$ is further processed by the safety shield described in Section 3.6.

3.6. Velocity-Projection Safety Shield

To reduce the collision risk caused by occasional high-risk actions generated by the neural policy, a velocity-projection safety shield is designed based on LiDAR sector ranges. Inspired by the safety-layer paradigm for continuous action spaces [30], the proposed safety shield acts on the policy output and reduces collision risk by suppressing excessive approaching-velocity components along high-risk LiDAR sector directions.

The safety shield takes the candidate velocity $\mathbf{v}_t^0$ from Section 3.5 as input, and the intermediate velocity is initialized as $\mathbf{v}_t^{(0)} = \mathbf{v}_t^0$. Let $\mathbf{n}_i$ denote the unit vector along the center direction of the i th LiDAR sector, and let $l_{t,i}$ denote the corresponding sector range. The velocity-dependent distance threshold is defined as

$$D_t = D_0 + k_v \|\mathbf{v}_t^0\|_2, \quad (10)$$

where $D_0 = 0.16$ m is the base distance threshold and $k_v = 0.12$ is the velocity-dependent gain. The LiDAR sectors are checked sequentially in ascending order of their range values, so sectors corresponding to closer obstacles are processed with higher priority. After a correction is triggered in one sector, the updated intermediate velocity is used for the

subsequent sector checks. For each LiDAR sector, the approaching-velocity component along the sector direction is computed as

$$q_i^{(m)} = \left(\mathbf{v}_t^{(m)} \right)^T \mathbf{n}_i, \quad (11)$$

where $\mathbf{v}_t^{(m)}$ denotes the intermediate velocity after m sector-wise corrections within the current control step. When $q_i^{(m)} \leq 0$ is non-positive, the velocity is not directed toward the obstacle along this sector, and no correction is applied. When $q_i^{(m)} > 0$ is positive, the maximum allowable approaching-velocity component is defined as

$$q_i^{\max} = \alpha \max(0, l_{t,i} - D_t), \quad (12)$$

where $\alpha = 1.8$ is the safety projection gain that converts the remaining distance margin into an upper bound on the approaching velocity. Therefore, a smaller sector range or a larger candidate speed reduces the allowable approaching velocity. When $l_{t,i} \leq D_t$, the allowable approaching component becomes zero, so the approaching-velocity component along this sector is suppressed. When $q_i^{(m)} > q_i^{\max}$, only the excessive approaching component is removed:

$$\Delta \mathbf{v}_i = \left(q_i^{(m)} - q_i^{\max} \right) \mathbf{n}_i, \quad (13)$$

$$\mathbf{v}_t^{(m+1)} = \mathbf{v}_t^{(m)} - \Delta \mathbf{v}_i. \quad (14)$$

If $q_i^{(m)} \leq q_i^{\max}$, no correction is triggered for that sector. During training, the accumulated correction magnitude is further used to construct the safety-shield correction reward, encouraging the policy to reduce high-risk actions. The same shield mechanism is also applied during testing to maintain consistency with training.

Figure 3 illustrates the sector-wise velocity-projection process used by the safety shield.

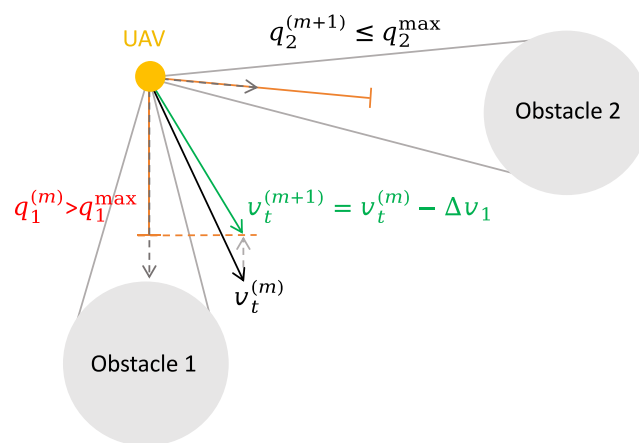


Figure 3. Illustration of the velocity-projection safety shield. The candidate velocity is projected onto LiDAR sector directions sequentially. In the lower-left sector, the approaching component exceeds the allowable bound and is therefore reduced along the sector direction, yielding the corrected velocity. In the upper-right sector, the approaching component of the corrected velocity remains within the allowable bound, so no further correction is triggered.

The collision threats addressed by the shield arise when a candidate command contains an excessive velocity component toward a nearby static or moving obstacle. Such commands may result from partial observability, short-term policy errors, or rapid changes in the local obstacle configuration. The risk may be associated with a single obstacle along the commanded direction or with multiple nearby obstacles located in different direc-

tions around the UAV. The resulting collision-risk reduction relies on several operating conditions. Collision-relevant surfaces must be detected with sufficient advance distance, the pooled sector ranges and their representative directions must adequately approximate the nearest-obstacle boundaries, and the sensing–control update rate and low-level velocity tracking must be fast relative to changes in the local scene. The shield itself uses the current sector ranges and the candidate UAV speed; it does not explicitly estimate obstacle velocity or predict future obstacle trajectories. Its effectiveness may therefore be reduced when an obstacle is missed or detected late because of finite angular resolution, occlusion, or missing returns, or when a moving obstacle approaches faster than the available sensing and control response. In narrow passages or dense clutter, corrections from multiple sectors may also restrict several velocity components, producing conservative motion or stagnation. This behavior is more likely in concave or trap-like configurations, where local velocity correction alone may not provide a long-horizon escape direction. Accordingly, the shield is intended to reduce local collision risk rather than replace global navigation or long-horizon planning.

Since the safety-shield parameters act directly on the allowable approaching-velocity bound, their influence is discussed through a local analytical sensitivity analysis. For each LiDAR sector, this bound limits the velocity component of the UAV toward the corresponding obstacle direction. A larger remaining clearance permits a larger approaching velocity, whereas a smaller clearance makes the shield more restrictive.

In the active-clearance region $l_{t,i} - D_t > 0$, holding the measured sector range $l_{t,i}$ and candidate speed $\|\mathbf{v}_t^0\|_2$ fixed gives the following local partial derivatives:

$$\frac{\partial q_i^{\max}}{\partial D_0} = -\alpha, \quad \frac{\partial q_i^{\max}}{\partial k_v} = -\alpha \|\mathbf{v}_t^0\|_2, \quad \frac{\partial q_i^{\max}}{\partial \alpha} = l_{t,i} - D_t. \quad (15)$$

These derivative relationships indicate the expected monotonic effects of the three parameters in the active-clearance region. Increasing D_0 enlarges the base protected distance and reduces the allowable approaching velocity, thereby making the shield intervene earlier. Increasing k_v also reduces the allowable approaching velocity, and this effect becomes stronger as the candidate speed $\|\mathbf{v}_t^0\|_2$ increases. In contrast, increasing α raises the allowable approaching velocity when $l_{t,i} - D_t > 0$ and therefore makes the shield less conservative in regions with positive clearance. Based on this trade-off, D_0 , k_v , and α were selected through preliminary simulation-based tuning and were then kept fixed during both training and testing across all scenarios to maintain a consistent evaluation protocol.

3.7. Reward Function

Single-step reward refers to the immediate reward assigned at each reinforcement learning decision step. In this study, the single-step reward is designed as a combination of six components: goal progress, obstacle clearance, velocity stability, safety-shield correction, action smoothness, and time/stagnation.

The numerical coefficients and thresholds in the reward function were manually set through empirical pilot tuning in simulation. They were selected according to the simulation scale, UAV speed limit, LiDAR sensing range, obstacle size, and relative magnitudes of different reward components. The same set of coefficients and thresholds was used for all training, ablation, and testing experiments.

The base reward at each decision step is defined as:

$$r_t = r_{\text{prog}} + r_{\text{obs}} + r_{\text{stab}} + r_{\text{shield}} + r_{\text{act}} + r_{\text{time}}. \quad (16)$$

If the UAV reaches the goal or collides with an obstacle at the current step, an additional terminal reward is added to the base reward:

$$R_t = \begin{cases} r_t + R_{\text{goal}}, & \text{goal reached,} \\ r_t + R_{\text{col}}, & \text{collision,} \\ r_t, & \text{otherwise,} \end{cases} \quad (17)$$

where $R_{\text{goal}} = 200$ and $R_{\text{col}} = -150$ are the terminal goal-reaching reward and terminal collision reward, respectively; R_{col} is a negative reward. The reward components are detailed below.

(1) Goal-progress reward. This component encourages progress toward the goal by rewarding reductions in the Euclidean goal distance between adjacent decision steps. It serves as the primary task-driving reward and is defined as:

$$r_{\text{prog}} = k_p(d_{t-1} - d_t), \quad (18)$$

$$d_t = \|\mathbf{p}_{gt}^{\text{rel}}\|_2, \quad (19)$$

where $k_p = 120$. This reward is positive when the UAV moves closer to the goal and negative when it moves away from it.

(2) Obstacle-safety reward. This component assigns a negative reward to unsafe clearance conditions only when the UAV enters a dangerous-clearance region. It discourages flight too close to obstacles and traversal through excessively narrow gaps. Let the range of the j th LiDAR sector at time step t be $l_{t,j}$, and let the RL decision period be Δt_{RL} . The two-frame LiDAR range variation rate is defined as:

$$\dot{l}_{t,j} = \frac{l_{t,j} - l_{t-1,j}}{\Delta t_{\text{RL}}}. \quad (20)$$

To improve sensitivity to approaching obstacles, a short-term predicted range is estimated over a fixed prediction window $\tau = 0.45$ s:

$$\hat{l}_{t,j} = l_{t,j} + \min(\dot{l}_{t,j}, 0)\tau. \quad (21)$$

The effective clearance at the current step is defined as the minimum over all current and predicted sector ranges:

$$c_t = \min_j \left(\min(l_{t,j}, \hat{l}_{t,j}) \right). \quad (22)$$

The obstacle-safety reward is formulated with logarithmic compression as:

$$r_{\text{obs}} = k_o \log \left[\text{clip} \left(\frac{c_t}{D_{\text{obs}}}, \varepsilon_s, 1 \right) \right], \quad (23)$$

where $D_{\text{obs}} = 0.7$ m is the obstacle-risk distance threshold, ε_s is the lower bound used to prevent divergence of the logarithmic term, and $k_o = 0.8$. When $c_t \geq D_{\text{obs}}$, the clipping result is one and $r_{\text{obs}} = 0$; when $c_t < D_{\text{obs}}$, the reward becomes negative, and smaller clearance values produce stronger negative rewards.

(3) Velocity-stability reward. This component assigns a negative reward to abrupt velocity variations and unstable behaviors, such as sudden stops, sudden accelerations, and unnecessary vertical oscillations. Let $\mathbf{v}_t$ denote the current velocity and $\mathbf{v}_{t-1}$ denote the previous velocity. This reward is defined as:

$$r_{\text{stab}} = k_s m_{\text{stab}}, \quad (24)$$

$$m_{\text{stab}} = \|\mathbf{v}_{t,xy} - \mathbf{v}_{t-1,xy}\|_2 + w_z |\mathbf{v}_{t,z} - \mathbf{v}_{t-1,z}|, \quad (25)$$

where $w_z = 1.2$ and $k_s = -0.12$. Larger velocity changes produce more negative reward values, and the vertical component is weighted more heavily to suppress unnecessary ascent and descent.

(4) Safety-shield correction reward. This component feeds the safety-shield correction magnitude back into policy training as a negative reward. Let the velocity correction generated by the i th sector be $\Delta \mathbf{v}_i$, and let $\mathcal{I}_t$ denote the set of sectors that trigger correction at time step t . The accumulated correction magnitude is:

$$M_{\text{sh}} = \sum_{i \in \mathcal{I}_t} \|\Delta \mathbf{v}_i\|_2, \quad (26)$$

$$r_{\text{shield}} = k_{\text{sh}} M_{\text{sh}}, \quad (27)$$

where $k_{\text{sh}} = -0.35$. A larger accumulated correction magnitude produces a more negative reward value. This design encourages the policy to generate lower-risk actions, reduce its reliance on the safety shield, and gradually shift from passive correction to active avoidance.

(5) Action-smoothness reward. This component assigns a negative reward to large magnitude jumps and abrupt direction reversals in the raw policy outputs between adjacent decision steps. Let the raw policy action be $\mathbf{a}_t$ and the previous action be $\mathbf{a}_{t-1}$. The magnitude change is measured using a dead-zone smoothness metric:

$$m_{\Delta a} = \max(0, \|\mathbf{a}_t - \mathbf{a}_{t-1}\|_2 - \delta_a). \quad (28)$$

The cosine similarity between adjacent action directions is defined as:

$$s_t = \frac{\mathbf{a}_t^T \mathbf{a}_{t-1}}{\|\mathbf{a}_t\|_2 \|\mathbf{a}_{t-1}\|_2 + \varepsilon_a}. \quad (29)$$

The reverse-switching metric is defined as:

$$m_{\text{rev}} = \max(0, -s_t). \quad (30)$$

The combined action-smoothness reward is:

$$r_{\text{act}} = k_{\Delta a} m_{\Delta a} + k_{\text{rev}} m_{\text{rev}}, \quad (31)$$

where $k_{\Delta a} = -0.08$ and $k_{\text{rev}} = -0.06$; $\delta_a = 0.05$ is the action-change dead-zone threshold, and ε_a is a small constant for numerical stability. The dead zone avoids assigning negative rewards to small action changes, while the reverse-switching metric is activated only when the cosine similarity is negative, corresponding to direction changes larger than 90° . This reward component mainly assigns negative rewards to large action jumps and reverse switching between adjacent actions, thereby reducing high-frequency control oscillations near obstacles.

(6) Time-and-stagnation reward. This component encourages efficient goal reaching while assigning a negative reward to persistent low-speed stagnation before the UAV enters the near-goal region. It is defined as:

$$r_{\text{time}} = k_{\text{time}} + r_{\text{hover}}, \quad (32)$$

where $k_{\text{time}} = -0.1$ is the fixed per-step time reward, which is negative. The stagnation reward r_{hover} is defined as:

$$r_{\text{hover}} = \begin{cases} k_{\text{hover}}, & \|\mathbf{p}_{g,t}^{\text{rel}}\|_2 > D_{\text{near}} \text{ and } \|\mathbf{v}_t\|_2 < v_{\text{hover}}, \\ 0, & \text{otherwise,} \end{cases} \quad (33)$$

where $k_{\text{hover}} = -0.1$ is the stagnation reward coefficient, which is negative, $D_{\text{near}} = 0.45$ m is the near-goal distance threshold, and $v_{\text{hover}} = 0.12$ m/s is the low-speed threshold. When the UAV has not entered the near-goal region, and its speed is below v_{hover} , an additional negative reward is applied. After the UAV enters the near-goal region, this negative reward is disabled to avoid discouraging normal deceleration near the goal.

4. Results

4.1. Simulation Environment and Parameters

All experiments were conducted on an Ubuntu 20.04 LTS system (Canonical Ltd., London, UK) equipped with an Intel Core i5-14600KF CPU and an NVIDIA GeForce RTX 5060 Ti GPU. The simulation environment was built using Gym-PyBullet-Drones [35], a PyBullet-based UAV simulation platform that integrates the Gym interface, the PyBullet physics engine, and multirotor control interfaces. Compared with visually oriented simulators, such as AirSim [36] and Flightmare [37], which are commonly used for photo-realistic rendering and perception-oriented tasks, this study does not rely on high-fidelity visual rendering. Therefore, Gym-PyBullet-Drones was adopted as a relatively lightweight PyBullet-based platform to reduce experimental complexity and facilitate frequent ablation studies and iterative parameter tuning [35,38].

The main simulation parameters are summarized in Table 1. The flight altitude was constrained within a narrow range, the start altitude was randomly sampled within this range, and the goal altitude was fixed. Walls were placed outside the core operating region. The obstacle height was fixed, and the obstacle radius was randomly sampled within a predefined range. At the beginning of each episode, the initial positions and horizontal velocity components of the dynamic obstacles were randomly sampled. The sampled initial positions were constrained to avoid overlap with static obstacles, other dynamic obstacles, and regions around the start and goal positions. The dynamic obstacles then moved independently with constant horizontal velocities. When an obstacle reached the boundary of the core operating region, the corresponding velocity component was reflected to keep the obstacle inside the workspace. Except for these boundary corrections, the obstacle velocities remained constant during motion. To evaluate the obstacle-avoidance performance of the UAV under different dynamic obstacle speeds, two speed settings were considered: a standard setting and a high-speed stress-test setting. These speed settings were selected according to the scale of the simulation task, the simulated quadrotor model used in Gym-PyBullet-Drones [35], the obstacle size, the sensing range, the control frequency, and the UAV motion constraints. In particular, the standard dynamic obstacle speed limit of 0.15 m/s corresponds to 25% of the maximum UAV command speed, while the high-speed setting of 0.35 m/s corresponds to approximately 58% of the maximum UAV command speed and was used to increase local collision risk.

For a fair comparison, fixed random seeds were used during testing, and all methods were evaluated using the same start points, goal points, obstacle distributions, and dynamic obstacle-motion settings.

The four mixed static–dynamic obstacle configurations are shown in Figure 4.

Table 1. Main simulation parameters.

Parameter	Value
Physical simulation frequency	240 Hz
Policy control frequency	30 Hz
Core operating region	8 m × 8 m
Flight/start altitude range	1.9–2.1 m
Goal altitude	2.0 m
Obstacle height	4.0 m
Obstacle radius	0.2–0.3 m
Maximum UAV speed	0.6 m/s
Dynamic obstacle speed limit	0.15/0.35 m/s
Maximum allowed steps per episode	1000 steps

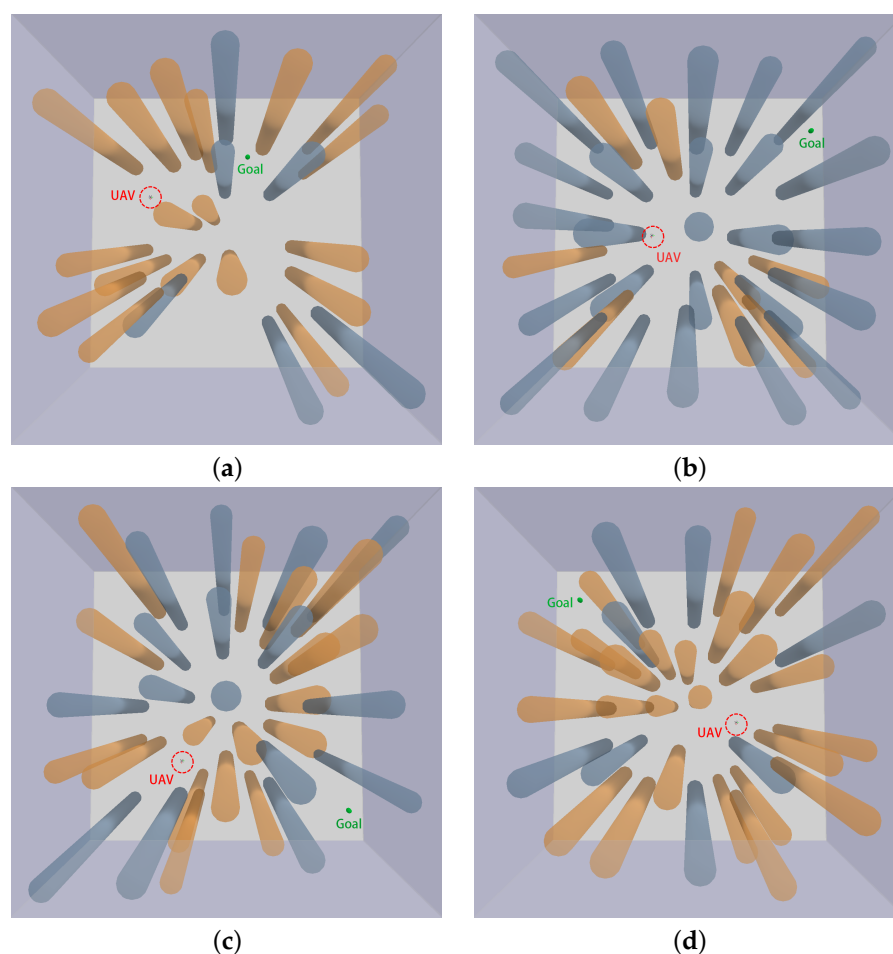


Figure 4. Mixed static–dynamic obstacle distributions in the simulation environment. Orange cylinders represent dynamic obstacles, and blue-gray cylinders represent static obstacles. The four configurations are: (a) 6 static and 18 dynamic obstacles, (b) 25 static and 8 dynamic obstacles, (c) 16 static and 17 dynamic obstacles, and (d) 8 static and 25 dynamic obstacles.

The four obstacle configurations were designed to represent different combinations of static–dynamic obstacle density. Configuration (a), with six static and 18 dynamic obstacles, represents a dynamic obstacle-dominated setting with relatively sparse static clutter. Configuration (b), with 25 static and eight dynamic obstacles, emphasizes dense static clutter with fewer moving obstacles. Configuration (c), with 16 static and 17 dynamic obstacles, provides a balanced mixed static–dynamic setting. Configuration (d), with eight static and 25 dynamic obstacles, represents a dynamic obstacle-rich setting, and the dynamic obstacle speed is further increased in the high-speed test to raise the collision risk. These

configurations were used to evaluate how different static–dynamic obstacle distributions affect obstacle-avoidance performance.

4.2. Curriculum Learning and Training Procedure

A staged curriculum learning strategy was used during training. The training procedure includes two auxiliary strategies: early-stage goal guidance and curriculum learning. The goal-guidance strategy provides an auxiliary velocity component pointing toward the goal during the early training stage. Its purpose is to reduce inefficient random exploration when the policy is still poorly trained and to help the UAV first acquire basic goal-directed navigation behavior. The influence of this guidance term is gradually reduced during training, and the policy performs independently without goal guidance during testing.

The curriculum learning strategy controls the order and difficulty of the training scenarios. Instead of exposing the policy directly to dense mixed static–dynamic environments at the beginning of training, the task difficulty is increased progressively. In the first stage, with the assistance of the goal-guidance term, the UAV learns goal-directed navigation and basic obstacle avoidance in fixed static-obstacle scenarios. In the second stage, training is performed in randomized obstacle scenarios, where the number of static and dynamic obstacles is gradually increased, while the policy-weight coefficient is increased and the goal-guidance weight is reduced. In the final stage, the training distribution is shifted toward dynamic obstacle-dominated scenarios, and the policy performs obstacle avoidance independently without guidance. This staged design is intended to improve training stability and allow the policy to acquire basic navigation, static-obstacle avoidance, and dynamic obstacle-avoidance behaviors progressively.

The main training hyperparameters are summarized in Table 2.

Table 2. Training hyperparameters.

Parameter	Value
Discount factor	0.99
Generalized advantage estimation (GAE) parameter	0.95
Clipping range	0.2
Rollout steps per update	19,200
Learning rate (initial)	2.5×10^{-4}
Entropy coefficient	0.015
Batch size	256
Value-function coefficient	0.5
Maximum gradient norm	0.5

Figure 5 compares the training success-rate curves under three training settings: the complete training strategy with both goal guidance and curriculum learning, a variant without goal guidance, and a variant without curriculum learning. The proposed goal-guidance and curriculum learning setting reaches a high success rate earlier than the ablation settings, suggesting faster convergence and improved training efficiency. The blue curve shows a temporary decrease approximately between 150,000 and 300,000 training steps, whereas the red curve shows a temporary decrease approximately between 250,000 and 400,000 training steps. These temporary decreases are associated with the curriculum-stage switch, after which the training scenarios change from fixed obstacle configurations to randomized obstacle configurations. As a result, the policy encounters more diverse obstacle layouts, which increases the training difficulty and leads to a short-term decrease in the success rate. After further training, the policy gradually adapts to the randomized scenarios, and the success rate increases again. Compared with the setting without curriculum learning, the curriculum-based settings still reach a high success rate earlier, indicating

that curriculum learning improves training efficiency despite the temporary performance drop during the difficulty transition.

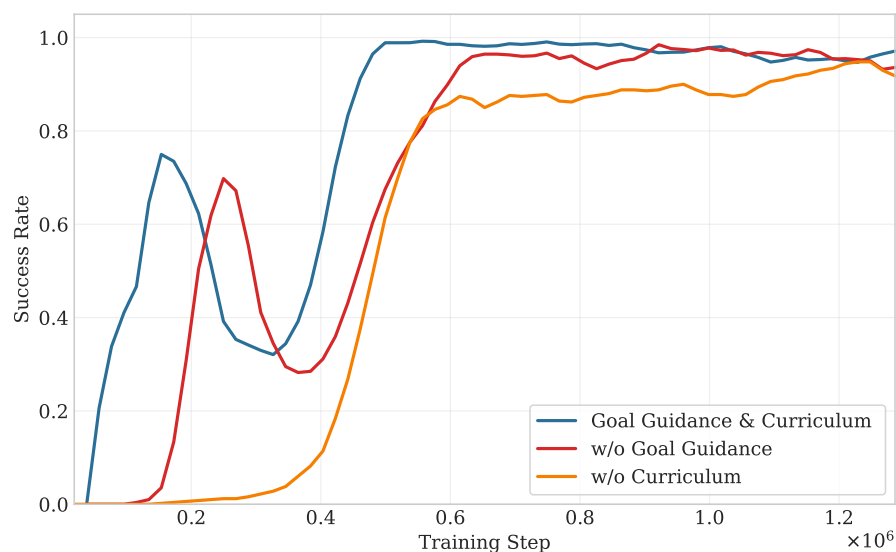


Figure 5. Training success-rate comparison under different training strategies. The proposed setting combines goal guidance and curriculum learning, whereas the two ablation settings remove goal guidance and curriculum learning, respectively. The faster rise and higher plateau of the proposed setting suggest that goal guidance and curriculum learning improve early-stage exploration and training efficiency.

Seven representative saved checkpoints were selected during training and evaluated under the same testing protocol. Figures 6–8 show the test success rate, average number of safety-shield activations, and average safety-shield correction reward of these checkpoints, respectively. The test success rate is low at the first two checkpoints, increases sharply at the third checkpoint, and remains close to 100% in subsequent checkpoints. The number of shield activations first increases as the policy begins to complete more effective flight interactions, and then decreases as the policy learns to generate lower-risk actions. Similarly, the safety-shield correction reward becomes less negative after the early low-reward checkpoints. These results suggest that the policy progressively reduces its reliance on safety-shield correction during training.

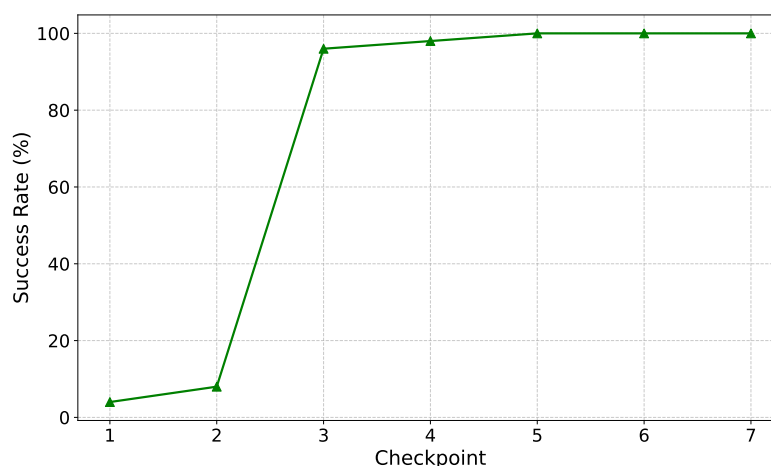


Figure 6. The test success rate of representative saved checkpoints during training. The increase in the success rate after the early checkpoints suggests improved navigation reliability as training progresses.

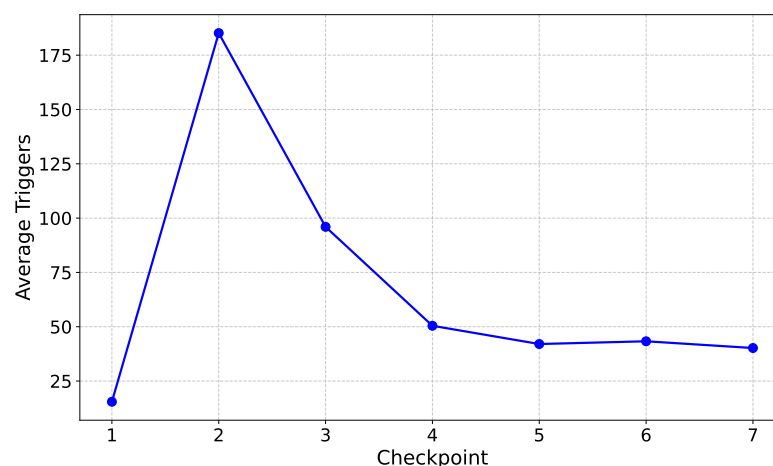


Figure 7. Average number of safety-shield activations for representative saved checkpoints. A shield activation indicates that the velocity-projection safety shield corrected a potentially risky action. The decreasing trend after the early checkpoints suggests that the learned policy gradually reduces its reliance on shield correction.

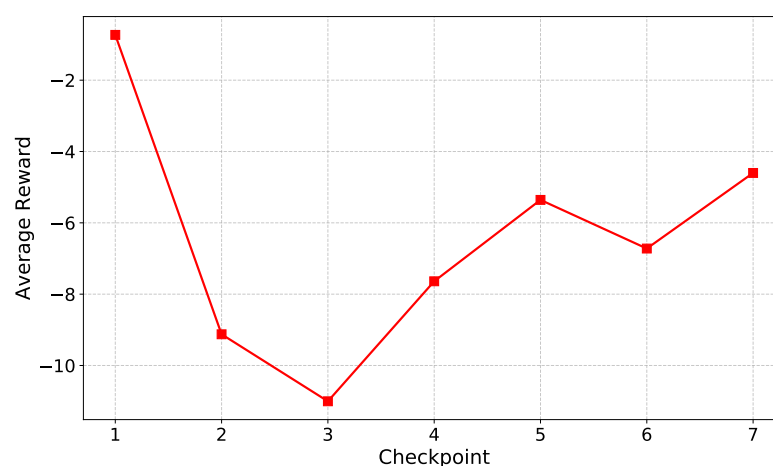


Figure 8. Average safety-shield correction reward for representative saved checkpoints. The correction reward is negative, and values closer to zero indicate smaller accumulated safety-shield corrections. The later checkpoints show less negative correction rewards, suggesting that the policy tends to generate lower-risk actions during training.

4.3. Ablation Experiments

To verify the contribution of each key module, three ablation variants were designed: without frame stacking, without LSTM, and without the safety shield. Figures 9 and 10 report results under three test scenarios: six static/18 dynamic obstacles, eight static/25 dynamic obstacles, and eight high-speed static/25 dynamic obstacles. For each scenario setting, the reported value is the mean $\pm$ standard deviation calculated from three different test scenario instances, with 500 randomized test episodes in each instance. The variants remove the two-frame short-term temporal cues, LSTM-based temporal memory, and safety-shield correction mechanism, respectively. In the variant without the safety shield, both the shield module and the corresponding safety-shield correction reward were removed during training and testing.

As shown in Figures 9 and 10, the complete method achieves the highest mean success rate across all three ablation scenarios, and its advantage becomes more evident as the task difficulty increases. In the six static/18 dynamic scenario, the complete method reaches a success rate of 95.60%, exceeding the ablation variants by 2.07–2.47 percentage points. In the eight static/25 dynamic scenario, its advantage increases to 5.00–6.80 percentage points. Un-

der the high-speed eight static/25 dynamic setting, where the upper speed limit of dynamic obstacles is increased to 0.35 m/s, the complete method achieves a success rate of 81.07%, outperforming the variants without frame stacking, without LSTM, and without the safety shield by 10.07, 9.94, and 15.14 percentage points, respectively. These results indicate that temporal input representation, recurrent temporal modeling, and safety-shield correction all contribute to navigation reliability, with the safety shield providing the largest improvement under the most challenging conditions. Path length is computed only over successful episodes. Although some ablation variants yield shorter mean paths, their substantially lower success rates suggest that these trajectories are more direct but less reliable. Overall, the mean $\pm$ standard deviation results show that the complete method achieves the highest success rate among the evaluated variants while maintaining an acceptable path length.

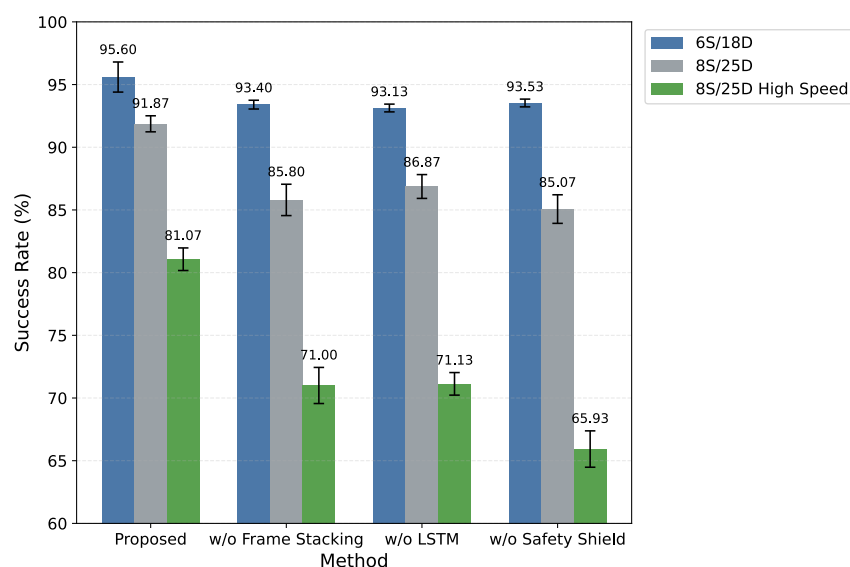


Figure 9. Mean $\pm$ standard deviation of success rate for the ablation experiments under different test scenarios. The complete method is compared with variants without frame stacking, without LSTM, and without the safety shield. The results indicate that the complete method achieves the highest mean success rate across the tested scenarios.

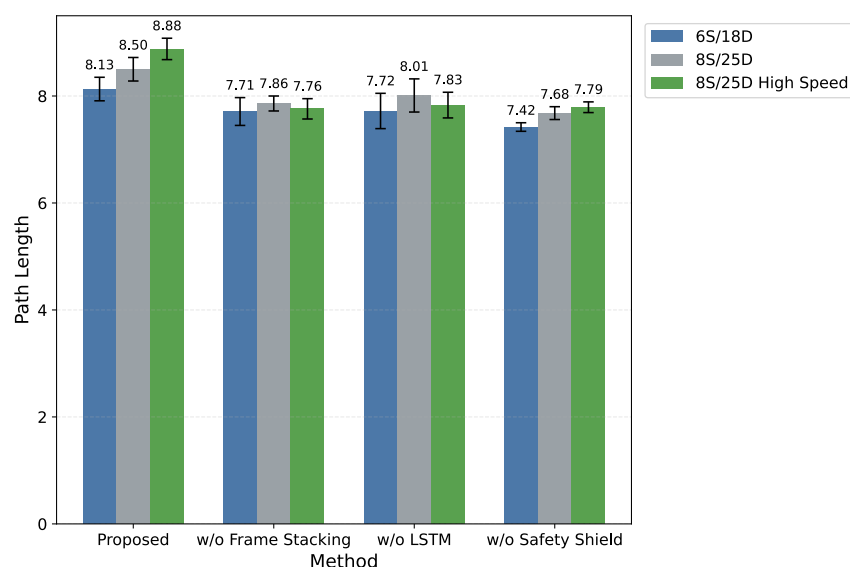


Figure 10. Mean $\pm$ standard deviation of path length for the ablation experiments under different test scenarios. Path length is computed only over successful episodes. The complete method yields slightly longer paths in some scenarios but maintains higher success rates, whereas some ablation variants produce shorter paths with lower success rates.

4.4. Baseline Comparison

The baselines include: (1) standard PPO [33], representing a basic deep reinforcement learning (DRL) baseline without the two-frame temporal observation, LSTM-based recurrent memory, or velocity-projection safety shield; (2) EGO-Planner [6], representing optimization-based local trajectory planning that generates dynamically feasible local trajectories according to the available obstacle information and UAV motion constraints; (3) Adaptive APF [39], representing an improved conventional reactive obstacle-avoidance method that computes the control command from goal-attractive and obstacle-repulsive terms based on the relative goal direction and local obstacle distances; and (4) RRT-DWA [40], representing a hybrid planning method in which a RRT generates a collision-free reference path from the start position to the goal position using the static-obstacle map, while DWA performs local velocity-space search to track the reference path and avoid nearby obstacles according to local LiDAR observations. The global RRT path is generated at mission start, and key sub-goal points selected from this path provide intermediate guidance for DWA local planning.

To ensure comparable external evaluation conditions, all methods were tested on the same hardware and under the same simulation scenarios, including identical start and goal positions, obstacle layouts, dynamic obstacle-motion settings, maximum speed limits, episode horizons, and success, collision, and timeout criteria. Each method used a fixed parameter configuration across the four test settings. However, because the methods have different information requirements and internal processing structures, identical internal representations were not imposed. Each method uses the environmental information required by its corresponding perception and planning pipeline. Specifically: the proposed method and PPO use local LiDAR observations; Adaptive APF uses local LiDAR observations; RRT-DWA uses local LiDAR observations and a static-obstacle map for global planning; and EGO-Planner uses a local point-cloud map.

For each test configuration, three independently generated scenario instances were used, and each instance contained 500 randomized test episodes. The reported metrics are presented as the mean $\pm$ standard deviation and include the success rate, collision rate, timeout rate, path length over successful episodes, and online computation time. Path length was computed only over successful episodes to measure trajectory efficiency under successful navigation.

Because the compared methods do not share the same computational structure, their online computation times are mainly determined by their respective online decision pipelines. For the proposed method and PPO, the timing covers one policy-network inference and action generation. For Adaptive APF, it covers one velocity-command computation based on the potential field. For EGO-Planner, it covers one local trajectory replanning process. For RRT-DWA, it covers one online local DWA planning process; the initial global RRT reference-path generation is performed at mission start and is excluded from the per-step timing. All timings were measured on the same hardware and excluded physics simulation, rendering, and low-level flight control. Therefore, the reported values are intended to reflect the online computational burden of each method.

The four obstacle scenarios reported in Table 3 represent different testing conditions. Static-obstacle-dominated settings mainly test whether a method can complete navigation in scenes with dense static clutter and constrained free space while avoiding excessive detours or timeout. Dynamic obstacle-dominated settings place greater emphasis on short-term obstacle-motion awareness, rapid local reaction, and collision-risk reduction. Therefore, variations in the reported evaluation metrics across the four configurations reflect not only overall performance but also the sensitivity of each method to different static–dynamic obstacle distributions.

As shown in Table 3, the proposed method achieved the highest average success rate among the evaluated methods across the four test configurations, with an average success rate of 91.88%, an average collision rate of 8.00%, and a near-zero average timeout rate of 0.12%. It obtained the highest success rate in the 25 static/eight dynamic, 16 static/17 dynamic, and high-speed eight static/25 dynamic scenarios. Compared with standard PPO, the proposed method increased the success rate by 9.80, 16.33, 19.20, and 22.54 percentage points in the four test configurations, respectively. These results suggest that, under the tested simulation settings, the two-frame LiDAR representation, LSTM-based temporal modeling, and safety-shield correction contribute to improved obstacle-avoidance stability.

Table 3. Baseline comparison under different static–dynamic obstacle distributions. The proposed method is compared with PPO, EGO-Planner, Adaptive APF, and RRT-DWA in terms of the success rate, collision rate, timeout rate, path length, and online computation time. Values are reported as the mean $\pm$ standard deviation over three independently generated scenario instances, with 500 randomized test episodes per instance. Path length is computed only over successful episodes. Bold values indicate the best result for each metric.

Scenario	Method	Success (%)	Collision (%)	Timeout (%)	Path Length (m)	Online Computation Time (ms)
6 S/18 D	Proposed	95.60 $\pm$ 1.20	4.40 $\pm$ 1.20	0.00 $\pm$ 0.00	8.13 $\pm$ 0.22	0.82 $\pm$ 0.05
	PPO [33]	85.80 $\pm$ 0.92	13.87 $\pm$ 0.64	0.33 $\pm$ 0.31	8.77 $\pm$ 0.10	0.49 $\pm$ 0.03
	EGO-Planner [6]	72.40 $\pm$ 2.03	27.60 $\pm$ 2.03	0.00 $\pm$ 0.00	7.97 $\pm$ 0.09	0.85 $\pm$ 0.01
	Adaptive APF [39]	80.73 $\pm$ 0.23	13.87 $\pm$ 0.92	5.40 $\pm$ 1.06	11.46 $\pm$ 0.13	0.51 $\pm$ 0.01
	RRT-DWA [40]	95.93 $\pm$ 0.64	1.93 $\pm$ 0.58	2.13 $\pm$ 0.12	9.15 $\pm$ 0.40	1.30 $\pm$ 0.02
25 S/8 D	Proposed	97.33 $\pm$ 0.23	2.53 $\pm$ 0.23	0.13 $\pm$ 0.23	8.17 $\pm$ 0.23	0.75 $\pm$ 0.02
	PPO [33]	81.00 $\pm$ 1.71	16.87 $\pm$ 2.30	2.13 $\pm$ 0.61	9.03 $\pm$ 0.20	0.50 $\pm$ 0.03
	EGO-Planner [6]	85.87 $\pm$ 1.92	14.13 $\pm$ 1.92	0.00 $\pm$ 0.00	7.96 $\pm$ 0.19	0.57 $\pm$ 0.01
	Adaptive APF [39]	41.33 $\pm$ 0.61	5.67 $\pm$ 0.76	53.00 $\pm$ 0.20	12.49 $\pm$ 0.31	0.46 $\pm$ 0.01
	RRT-DWA [40]	82.93 $\pm$ 1.50	0.67 $\pm$ 0.42	16.40 $\pm$ 1.91	9.89 $\pm$ 0.34	1.36 $\pm$ 0.02
16 S/17 D	Proposed	93.53 $\pm$ 0.12	6.13 $\pm$ 0.42	0.33 $\pm$ 0.31	8.62 $\pm$ 0.37	0.76 $\pm$ 0.01
	PPO [33]	74.33 $\pm$ 0.23	24.73 $\pm$ 0.70	0.93 $\pm$ 0.76	8.78 $\pm$ 0.35	0.49 $\pm$ 0.02
	EGO-Planner [6]	66.47 $\pm$ 1.75	33.53 $\pm$ 1.75	0.00 $\pm$ 0.00	8.03 $\pm$ 0.12	0.75 $\pm$ 0.09
	Adaptive APF [39]	39.80 $\pm$ 0.60	6.40 $\pm$ 1.00	53.80 $\pm$ 0.87	12.67 $\pm$ 0.30	0.47 $\pm$ 0.01
	RRT-DWA [40]	88.40 $\pm$ 1.83	3.80 $\pm$ 0.87	7.80 $\pm$ 1.11	10.28 $\pm$ 0.27	1.36 $\pm$ 0.02
8 S/25 D high-speed	Proposed	81.07 $\pm$ 0.90	18.93 $\pm$ 0.90	0.00 $\pm$ 0.00	8.88 $\pm$ 0.20	0.77 $\pm$ 0.01
	PPO [33]	58.53 $\pm$ 1.30	41.47 $\pm$ 1.30	0.00 $\pm$ 0.00	9.13 $\pm$ 0.13	0.53 $\pm$ 0.01
	EGO-Planner [6]	37.07 $\pm$ 0.70	62.93 $\pm$ 0.70	0.00 $\pm$ 0.00	7.93 $\pm$ 0.48	0.84 $\pm$ 0.04
	Adaptive APF [39]	16.93 $\pm$ 1.63	82.00 $\pm$ 1.06	1.07 $\pm$ 0.58	10.36 $\pm$ 0.28	0.60 $\pm$ 0.01
	RRT-DWA [40]	78.47 $\pm$ 1.22	20.67 $\pm$ 1.42	0.87 $\pm$ 0.50	10.75 $\pm$ 0.14	1.43 $\pm$ 0.03

Compared with EGO-Planner, which is based on local trajectory optimization, the proposed method showed higher success rates in scenarios with a higher proportion or higher speed of dynamic obstacles. Although EGO-Planner achieved relatively short successful path lengths, its average success rate decreased in dynamic obstacle-dominated scenarios, especially in the high-speed eight static/25 dynamic scenario. PPO and Adaptive APF had low online computation times, but their average success rates were lower; Adaptive APF also produced higher timeout rates, especially in the 25 static/eight dynamic and 16 static/17 dynamic scenarios. These results suggest that, in the tested dynamic simulation scenarios, standard PPO and Adaptive APF may still have certain limitations in simultaneously maintaining low computational cost and stable obstacle-avoidance behavior.

RRT-DWA achieved high success rates in most scenarios, with an average success rate of 86.43%. For the six static/18 dynamic scenario, the success rates reported in Table 3 are close for the proposed method and RRT-DWA, with values of 95.60 $\pm$ 1.20% and 95.93 $\pm$ 0.64%, respectively. Because RRT-DWA achieved the highest average success rate among the baselines and its success rate was closest to that of the proposed method in the six

static/18 dynamic scenario, a two-sided exact McNemar test was conducted using matched episode-level success/failure outcomes to avoid over-interpreting this small numerical difference. The discordant outcomes were balanced, with 58 episodes successful only for the proposed method and 63 episodes successful only for RRT-DWA. The test showed no statistically significant difference in the success rate between the two methods in this scenario ($p = 0.7163$) [41]. Therefore, this study does not draw a superiority conclusion regarding the success rate of either method in this individual scenario. However, RRT-DWA produced longer successful paths. Under the timing definitions stated above, RRT-DWA and the proposed method had average online computation times of approximately 1.36 ms and 0.78 ms, respectively. The average path length over successful episodes of the proposed method was 8.45 m, close to the 7.97 m of EGO-Planner and shorter on average than those of Adaptive APF and RRT-DWA.

Overall, the proposed method achieved the highest average success rate among the evaluated methods while maintaining a low timeout rate, competitive path length, and relatively short online computation time. These results suggest a reasonable balance among avoidance reliability, path efficiency, and online computational cost under the tested simulation conditions.

Figure 11 provides representative trajectory comparisons of different algorithms under the same start and goal, obstacle layout, and dynamic obstacle-motion settings. In these selected cases, the proposed method produces relatively smooth collision-free trajectories, whereas several baseline methods show detours, oscillatory motions, or collision failures.

A magnified view of a local turning maneuver of the proposed method is provided in Figure 12. The zoomed region illustrates how the UAV adjusts its trajectory when passing through nearby obstacles.

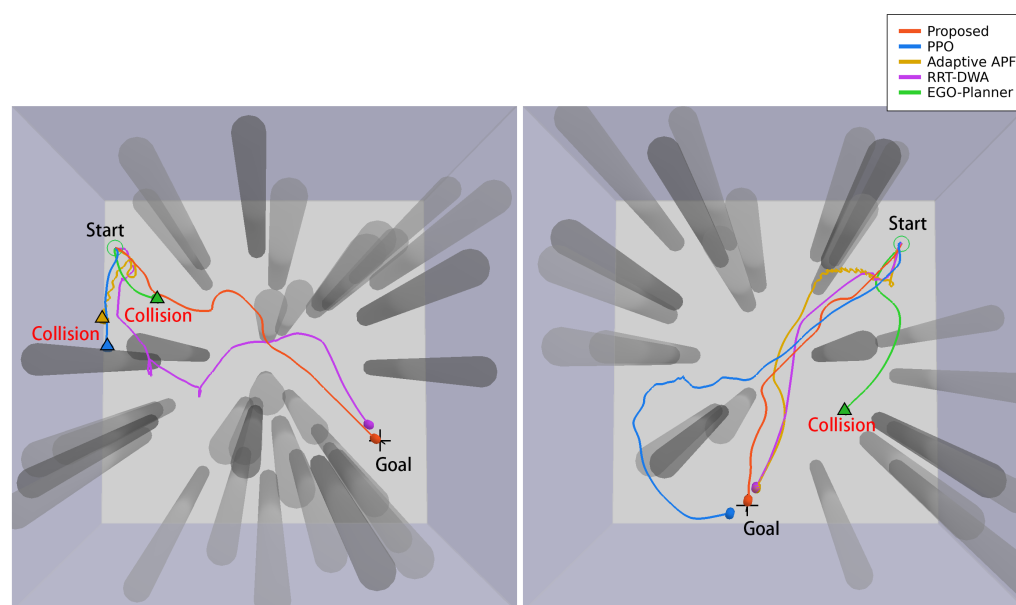


Figure 11. Representative trajectory comparison of different algorithms under the same start and goal, obstacle layout, and dynamic obstacle-motion settings. Obstacles are shown in gray, and triangles indicate collision points. The trajectories of the proposed method and baseline methods are shown with different colors. The comparison shows that the proposed method generates collision-free trajectories in the selected representative cases, whereas some baseline methods exhibit detours, oscillatory motions, or collision failures.

Figure 13 provides an exploratory qualitative example of a possible extension scenario in which the goal position changes over time in a dynamic obstacle environment. In this example, the UAV adjusts its trajectory according to the changing goal position while

avoiding surrounding obstacles. This example is intended to show a possible task scenario in which the proposed framework may be further extended from fixed-goal obstacle avoidance to moving-goal navigation. However, this study mainly focuses on fixed-goal UAV obstacle avoidance, and systematic quantitative evaluation under moving-goal conditions will be investigated in future work.

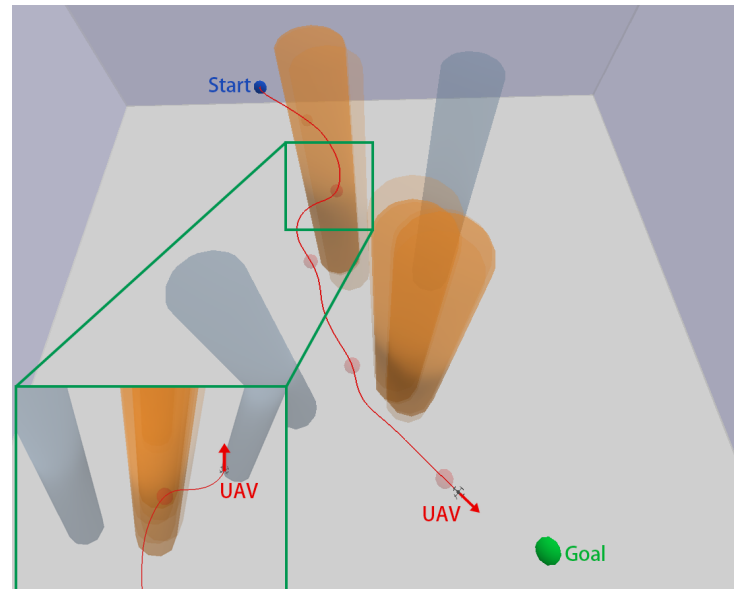


Figure 12. Detailed view of the local obstacle-avoidance behavior of the proposed method. Orange cylinders represent dynamic obstacles, and blue-gray cylinders represent static obstacles. The green box indicates the zoomed region. The red curve denotes the UAV trajectory, and the arrows indicate the UAV's motion direction. The enlarged view highlights a local turning maneuver near surrounding obstacles.

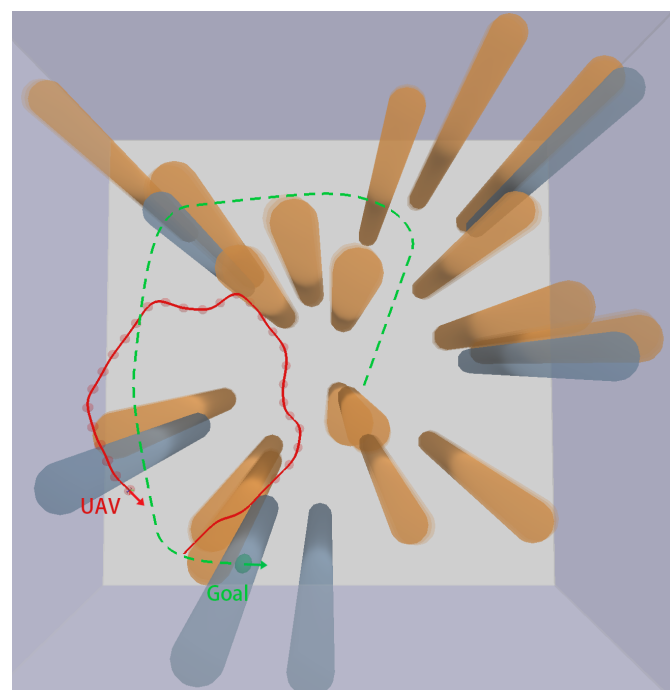


Figure 13. Exploratory qualitative example of a possible moving-goal extension scenario in a dynamic obstacle environment. Orange cylinders represent dynamic obstacles, and blue-gray cylinders represent static obstacles. The red curve denotes the UAV trajectory, and the green dashed curve denotes the moving-goal trajectory. The labels and arrows indicate the UAV and moving-goal positions and their motion directions in this representative example.

5. Discussion

The results suggest that combining temporal LiDAR representation, recurrent temporal modeling, and safety-shield-based action correction is associated with improved UAV obstacle-avoidance reliability in the tested mixed static–dynamic simulation environments. Compared with the standard PPO baseline, the proposed method achieved higher success rates across the tested configurations. This observation suggests that short-term temporal cues are useful for dynamic obstacle avoidance, rather than relying solely on a single instantaneous range observation. In local avoidance tasks, a single LiDAR scan describes only the current spatial distribution of obstacles, whereas consecutive observations and recurrent hidden states can provide additional information about local range variation and short-term collision tendency.

The two-frame LiDAR representation and the LSTM module appear to contribute in different but complementary ways. Sector-wise minimum pooling preserves nearest-obstacle information in a compact form, and two-frame stacking makes short-term range changes explicitly available to the policy. Such temporal information can help differentiate approaching, receding, and approximately static obstacles in the tested scenarios. The LSTM further aggregates historical observations, which can reduce the reliance on only instantaneous observations under partial observability. The performance degradation observed after removing either frame stacking or the LSTM was more pronounced in challenging dynamic obstacle scenarios. However, the temporal representation remains a short-horizon reactive mechanism and does not explicitly predict long-term obstacle trajectories or perform global path reasoning. As a result, scenarios that require long-horizon reasoning, such as dead-end or U-shaped obstacle configurations, may still require additional exploration mechanisms.

The safety shield contributes to collision-risk reduction in the tested scenarios, particularly in the high-speed dynamic obstacle setting. A purely learned policy may occasionally output high-risk commands near obstacles, whereas the velocity-projection shield is designed to suppress excessive approaching components along LiDAR sector directions. By including the correction magnitude in the reward, the policy is encouraged to rely less on shield intervention during training. This mechanism should be viewed as an empirical correction strategy for high-risk actions rather than an absolute guarantee of action safety. Its effectiveness depends on LiDAR sensing quality, update frequency, obstacle geometry, and the tracking performance of the low-level controller. Collision risk may still exist under severe sensing noise, occlusion, control delay, or fast obstacle motion outside the sensing horizon.

The baseline comparison suggests that the proposed method maintains a reasonable balance among avoidance reliability, path efficiency, and online computation time under the tested simulation conditions. In dynamic obstacle-rich scenarios, the proposed framework achieved higher success rates while maintaining a relatively short online computation time. These results provide empirical support for using lightweight temporal perception and velocity-projection action correction for local reactive avoidance under limited onboard computation.

Nevertheless, it should be noted that the current evaluation is still limited to simulated sensing and control conditions. Although the ablation experiments provide module-level control comparisons by removing frame stacking, the LSTM module, and the safety shield, they do not fully account for real-world sensing and control uncertainties. In real-world UAV deployment, LiDAR measurements can be affected by range noise, missing returns, reflective surfaces, limited angular resolution, and temporary occlusion. In addition, actuator delay, aerodynamic disturbances, and model mismatch between the simulator and the real platform can further affect avoidance performance. The level of performance degradation

during real-world adaptation is likely to depend on the severity and combination of these uncertainties. Under relatively mild sensing disturbances, the policy may still complete the task, but with more frequent safety-shield activations. Under more severe missing returns, occlusion, control delay, or dynamics mismatch, the degradation may become more pronounced and may appear as a reduced success rate, an increased collision rate, or an increased timeout rate. Therefore, the reported results should be interpreted as simulation-based evidence of the proposed framework rather than a quantitative estimate of real-world deployment performance. Future work will explore noise-aware obstacle-avoidance training and testing, with particular emphasis on training strategies that incorporate sensor noise, missing returns, observation uncertainty, and control delay under more realistic sensing and control conditions. Robustness on physical UAV platforms and in real-world environments will also be examined in future work.

In addition to noise-aware training and physical validation, future work may also strengthen the theoretical analysis of reinforcement learning-based UAV obstacle avoidance under sensing uncertainty, actuation errors, and control delays. Recent stability-analysis methods for time-delay systems, together with genetic-algorithm-based parameter optimization, may provide useful references for analyzing control-delay effects and robust parameter tuning [42]. Related studies on finite-time and fixed-time synchronization of fractional-order fuzzy neural networks with adaptive control may also provide theoretical perspectives for analyzing convergence and robustness properties of neural-network-based control systems [43,44]. These theories have not been incorporated into the present reinforcement learning-based UAV obstacle-avoidance framework, and their applicability to dynamic obstacle avoidance requires dedicated investigation in future work.

6. Conclusions

This study proposed a reinforcement learning-based local obstacle-avoidance framework for small quadrotor UAVs in mixed static–dynamic environments. The method integrates a two-frame multi-layer LiDAR representation, CNN-LSTM temporal feature extraction, Recurrent PPO policy optimization, and velocity-projection safety-shield action correction. Instead of explicitly detecting, tracking, or predicting individual dynamic obstacles, the proposed framework encodes local dynamic changes through consecutive LiDAR observations and recurrent hidden states while using the safety shield to reduce empirical collision risk during training and execution.

Simulation experiments in four mixed static–dynamic test configurations showed that the proposed method achieved an average success rate of 91.9% and an average online computation time of 0.78 ms. Ablation experiments supported the contributions of two-frame LiDAR observations, LSTM-based temporal modeling, and the safety shield. Baseline comparisons further showed that the proposed method achieved a reasonable balance among the success rate, path efficiency, and online computation time under the tested simulation conditions.

The current evaluation is limited to simulated sensing and control conditions, with constrained altitude variation and primarily horizontal avoidance behavior. It does not fully account for real-world LiDAR noise, missing returns, state-estimation errors, flight-control delay, aerodynamic disturbances, or differences between simulated UAV dynamics and real physical UAV behavior. Future work will focus on noise-aware training and testing, robustness evaluation under more realistic sensing and control uncertainties, physical UAV validation, and extension to scenarios with larger altitude variations and more diverse obstacle configurations.

Author Contributions: Conceptualization, C.L. and S.W.; methodology, C.L.; software, C.L.; validation, C.L.; formal analysis, C.L.; investigation, C.L.; resources, S.W.; data curation, C.L.; writing—original draft preparation, C.L.; writing—review and editing, C.L. and S.W.; visualization, C.L.; supervision, S.W.; project administration, S.W. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original contributions presented in this study are included in this article. Further inquiries can be directed to the corresponding author.

Acknowledgments: During the preparation of this manuscript, the authors used ChatGPT (OpenAI, GPT-5.5 Thinking; accessed May 2026) for language polishing, wording refinement, and clarity improvement. The authors have reviewed and edited the output and take full responsibility for the content of this publication.

Conflicts of Interest: The authors declare no conflicts of interest.

References

- Wang, T.; Yang, L.; Chang, Y.; Huang, Z.; Jiang, H.; Zheng, Y. A review of dynamic obstacle avoidance for unmanned aerial vehicles (UAVs). In Proceedings of the 2024 7th International Symposium on Autonomous Systems (ISAS), Chongqing, China, 7–9 May 2024; pp. 274–279. [\[CrossRef\]](#)
- Merei, A.; Mcheick, H.; Ghaddar, A.; Rebaine, D. A survey on obstacle detection and avoidance methods for UAVs. *Drones* **2025**, *9*, 203. [\[CrossRef\]](#)
- Xia, W.; Song, F.; Peng, Z. Dynamic obstacle perception technology for UAVs based on LiDAR. *Drones* **2025**, *9*, 540. [\[CrossRef\]](#)
- Xu, Z.; Jin, H.; Han, X.; Shen, H.; Shimada, K. Intent prediction-driven model predictive control for UAV planning and navigation in dynamic environments. *IEEE Robot. Autom. Lett.* **2025**, *10*, 4946–4953. [\[CrossRef\]](#)
- Memlikai, G.; Tsintotas, K.A. Reinforcement learning for UAV control: From algorithms to deployment readiness. *Machines* **2026**, *14*, 177. [\[CrossRef\]](#)
- Zhou, X.; Wang, Z.; Ye, H.; Xu, C.; Gao, F. EGO-Planner: An ESDF-free gradient-based local planner for quadrotors. *IEEE Robot. Autom. Lett.* **2021**, *6*, 478–485. [\[CrossRef\]](#)
- Xu, Z.; Xiu, Y.; Zhan, X.; Chen, B.; Shimada, K. Vision-aided UAV navigation and dynamic obstacle avoidance using gradient-based B-spline trajectory optimization. In Proceedings of the IEEE International Conference on Robotics and Automation (ICRA), London, UK, 29 May–2 June 2023; pp. 1214–1220. [\[CrossRef\]](#)
- Lu, M.; Fan, X.; Chen, H.; Lu, P. FAPP: Fast and adaptive perception and planning for UAVs in dynamic cluttered environments. *IEEE Trans. Robot.* **2025**, *41*, 871–886. [\[CrossRef\]](#)
- Tordesillas, J.; Lopez, B.T.; How, J.P. FASTER: Fast and safe trajectory planner for flights in unknown environments. In Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Macau, China, 3–8 November 2019; pp. 1934–1940. [\[CrossRef\]](#)
- Xu, Z.; Han, X.; Shen, H.; Jin, H.; Shimada, K. NavRL: Learning safe flight in dynamic environments. *IEEE Robot. Autom. Lett.* **2025**, *10*, 3668–3675. [\[CrossRef\]](#)
- Fan, X.; Lu, M.; Xu, B.; Lu, P. Flying in highly dynamic environments with end-to-end learning approach. *IEEE Robot. Autom. Lett.* **2025**, *10*, 3851–3858. [\[CrossRef\]](#)
- Liu, J.; Luo, W.; Zhang, G.; Li, R. Unmanned aerial vehicle path planning in complex dynamic environments based on deep reinforcement learning. *Machines* **2025**, *13*, 162. [\[CrossRef\]](#)
- Miera, P.; Szolc, H.; Kryjak, T. LiDAR-based drone navigation with reinforcement learning. *arXiv* **2023**, arXiv:2307.14313. [\[CrossRef\]](#)
- Kaufmann, E.; Bauersfeld, L.; Loquercio, A.; Müller, M.; Koltun, V.; Scaramuzza, D. Champion-level drone racing using deep reinforcement learning. *Nature* **2023**, *620*, 982–987. [\[CrossRef\]](#) [\[PubMed\]](#)
- Loquercio, A.; Kaufmann, E.; Ranftl, R.; Müller, M.; Koltun, V.; Scaramuzza, D. Learning high-speed flight in the wild. *Sci. Robot.* **2021**, *6*, eabg5810. [\[CrossRef\]](#) [\[PubMed\]](#)
- Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A.A.; Veness, J.; Bellemare, M.G.; Graves, A.; Riedmiller, M.; Fidjeland, A.K.; Ostrovski, G.; et al. Human-level control through deep reinforcement learning. *Nature* **2015**, *518*, 529–533. [\[CrossRef\]](#) [\[PubMed\]](#)

17. Song, X. Design and application of an intelligent decision-making system for unmanned aerial vehicles based on deep reinforcement learning. *IEEE Access* **2025**, *13*, 171435–171441. [\[CrossRef\]](#)
18. de Heuvel, J.; Zeng, X.; Shi, W.; Sethuraman, T.; Bennewitz, M. Spatiotemporal attention enhances lidar-based robot navigation in dynamic environments. *IEEE Robot. Autom. Lett.* **2024**, *9*, 4202–4209. [\[CrossRef\]](#)
19. Hausknecht, M.; Stone, P. Deep recurrent Q-learning for partially observable MDPs. *arXiv* **2015**, arXiv:1507.06527. [\[CrossRef\]](#)
20. Singla, A.; Padakandla, S.; Bhatnagar, S. Memory-based deep reinforcement learning for obstacle avoidance in UAV with limited environment knowledge. *IEEE Trans. Intell. Transp. Syst.* **2021**, *22*, 107–118. [\[CrossRef\]](#)
21. Hart, P.E.; Nilsson, N.J.; Raphael, B. A formal basis for the heuristic determination of minimum cost paths. *IEEE Trans. Syst. Sci. Cybern.* **1968**, *4*, 100–107. [\[CrossRef\]](#)
22. LaValle, S.M. *Rapidly-Exploring Random Trees: A New Tool for Path Planning*; Technical Report TR 98-11; Department of Computer Science, Iowa State University: Ames, IA, USA, 1998.
23. Khatib, O. Real-time obstacle avoidance for manipulators and mobile robots. *Int. J. Robot. Res.* **1986**, *5*, 90–98. [\[CrossRef\]](#)
24. Fiorini, P.; Shiller, Z. Motion planning in dynamic environments using velocity obstacles. *Int. J. Robot. Res.* **1998**, *17*, 760–772. [\[CrossRef\]](#)
25. Xu, G.; Wu, T.; Wang, Z.; Wang, Q.; Gao, F. Flying on point clouds with reinforcement learning. In Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Hangzhou, China, 19–25 October 2025; pp. 7231–7238. [\[CrossRef\]](#)
26. Xie, Z.; Dames, P. DRL-VO: Learning to navigate through crowded dynamic scenes using velocity obstacles. *IEEE Trans. Robot.* **2023**, *39*, 2700–2719. [\[CrossRef\]](#)
27. Xu, B.; Yan, Z.; Lu, M.; Fan, X.; Luo, Y.; Lin, Y.; Chen, Z.; Chen, Y.; Qiao, Q.; Lu, P. Flow-aided flight through dynamic clutters from point to motion. *IEEE Robot. Autom. Lett.* **2026**, *11*, 218–225. [\[CrossRef\]](#)
28. Luo, W.; Wang, X.; Han, F.; Zhou, Z.; Cai, J.; Zeng, L.; Chen, H.; Chen, J.; Zhou, X. Research on LSTM-PPO obstacle avoidance algorithm and training environment for unmanned surface vehicles. *J. Mar. Sci. Eng.* **2025**, *13*, 479. [\[CrossRef\]](#)
29. Song, S. LSTM-DDPG-based dynamic obstacle avoidance for UAVs in power distribution networks using velocity obstacle modeling. *Informatica* **2025**, *49*, 65–74. [\[CrossRef\]](#)
30. Dalal, G.; Dvijotham, K.; Vecerik, M.; Hester, T.; Paduraru, C.; Tassa, Y. Safe exploration in continuous action spaces. *arXiv* **2018**, arXiv:1801.08757. [\[CrossRef\]](#)
31. LeCun, Y.; Bottou, L.; Bengio, Y.; Haffner, P. Gradient-Based Learning Applied to Document Recognition. *Proc. IEEE* **1998**, *86*, 2278–2324. [\[CrossRef\]](#)
32. Greff, K.; Srivastava, R.K.; Koutník, J.; Steunebrink, B.R.; Schmidhuber, J. LSTM: A search space odyssey. *IEEE Trans. Neural Netw. Learn. Syst.* **2017**, *28*, 2222–2232. [\[CrossRef\]](#) [\[PubMed\]](#)
33. Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; Klimov, O. Proximal policy optimization algorithms. *arXiv* **2017**, arXiv:1707.06347. [\[CrossRef\]](#)
34. Narvekar, S.; Peng, B.; Leonetti, M.; Sinapov, J.; Taylor, M.E.; Stone, P. Curriculum learning for reinforcement learning domains: A framework and survey. *J. Mach. Learn. Res.* **2020**, *21*, 1–50.
35. Panerati, J.; Zheng, H.; Zhou, S.; Xu, J.; Prorok, A.; Schoellig, A.P. Learning to fly—A gym environment with PyBullet physics for reinforcement learning of multi-agent quadcopter control. In Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Prague, Czech Republic, 27 September–1 October 2021; pp. 7512–7519. [\[CrossRef\]](#)
36. Shah, S.; Dey, D.; Lovett, C.; Kapoor, A. AirSim: High-fidelity visual and physical simulation for autonomous vehicles. In *Field and Service Robotics*; Hutter, M., Siegwart, R., Eds.; Springer: Cham, Switzerland, 2018; pp. 621–635. [\[CrossRef\]](#)
37. Song, Y.; Naji, S.; Kaufmann, E.; Loquercio, A.; Scaramuzza, D. Flightmare: A flexible quadrotor simulator. In Proceedings of the Conference on Robot Learning (CoRL), Cambridge, MA, USA, 16–18 November 2020; pp. 1147–1157.
38. Nguyen-Duong-Hoang, P.; Phan-Van, T.; Pham-Ngoc, S.; Dang-Le-Bao, C.; Le-Trung, Q. IsaacLab vs. gym-pybullet-drones: A comparative study of UAV simulators for reinforcement learning. In Proceedings of the RIVF International Conference on Computing and Communication Technologies (RIVF), Ho Chi Minh City, Vietnam, 18–20 December 2025; pp. 179–184. [\[CrossRef\]](#)
39. Kilic, K.I.; Desoeuvres, A.; Pedersen, C.B.; Vasegaard, A.E.; Nielsen, P. Adaptive artificial potential field method for small autonomous vehicles. *Robot. Auton. Syst.* **2026**, *198*, 105364. [\[CrossRef\]](#)
40. Han, Q.; Ma, X.; Liu, J.; Liu, H.; Yan, Y.; Yang, Q. A hybrid RRT-DWA path planning framework for UAVs in dynamic environments. *Sci. Rep.* **2026**, *16*, 3089. [\[CrossRef\]](#) [\[PubMed\]](#)
41. McNemar, Q. Note on the Sampling Error of the Difference between Correlated Proportions or Percentages. *Psychometrika* **1947**, *12*, 153–157. [\[CrossRef\]](#) [\[PubMed\]](#)
42. Nie, Y.; Zhao, C.; Shi, K.; Chen, Y. Novel Stability Analysis for Time-Delay Systems via Two New Lemmas and Genetic Algorithm. *Chaos Solitons Fractals* **2026**, *208*, 118232. [\[CrossRef\]](#)

43. Fan, H.; Shi, K.; Guo, Z.; Zhou, A.; Cai, J. Finite-Time Synchronization and Mittag–Leffler Synchronization for Uncertain Fractional-Order Delayed Cellular Neural Networks with Fuzzy Operators via Nonlinear Adaptive Control. *Fractal Fract.* **2025**, *9*, 634. [[CrossRef](#)]
44. Fan, H.; Shi, K.; Zhou, A.; Meng, F.; Jiang, L. Exploring Fixed-Time Synchronization of Fractional-Order Fuzzy Cellular Neural Networks with Information Interactions and Time-Varying Delays via Adaptive Multi-Module Control. *Fractal Fract.* **2026**, *10*, 253. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.